Academy Hub — Sesión 6

Apps pendientes: modelos, CRUD, permisos y flujos completos

> **Nota:** esta sesión extiende el proyecto con las 8 apps que faltaban para cubrir el ciclo completo de Academy Hub tal y como se diseñó originalmente. Se aplican los mismos patrones de la Sesión 5: `ModelForm`, Class Based Views, `LoginRequiredMixin`, `UserPassesTestMixin`, object-level permissions, `select_related`, `prefetch_related`, paginación, mensajes y logging.

Índice

1. Objetivo general
2. Convenciones del proyecto (recordatorio)
3. Mapa de permisos por rol y entidad
4. App `reviews`
5. App `payments`
6. App `notifications`
7. App `evaluations`
8. App `talent`
9. App `level_tests`
10. App `search`
11. App `dashboard`
12. Registro en settings y urls
13. Tabla de URLs finales
14. Tabla de permisos final
15. Checklist global
16. Orden de implementación recomendado
17. Commit recomendado
18. Notas de integración con apps existentes

---

1. Objetivo general

Completar Academy Hub implementando las 8 apps que faltan para cubrir todo el ciclo del alumno y de la academia:

- Descubrir cursos.
- Reservar y matricularse.
- Pagar.

- Recibir recordatorios.
- Hacer prueba de nivel.
- Ser evaluado.
- Construir el Pasaporte de Talento.
- Valorar cursos y academias.

Al terminar, la plataforma tendrá:

- Un alumno con historial completo.
- Una academia con gestión integral.
- Un buscador avanzado.
- Un dashboard por rol.

---

2. Convenciones del proyecto (recordatorio)

- `app_name` en cada `urls.py`, nombres de URL en inglés y snake_case.
- Rutas con `pk` + `slug` cuando la entidad tiene slug.
- `get_absolute_url()` en modelos con vista pública.
- `select_related` para FK, `prefetch_related` para 1:N y M:N.
- `LoginRequiredMixin` + `UserPassesTestMixin` para permisos.
- Mensajes con `messages` y logging con `logger.info` en acciones críticas.
- Templates en `templates/<app>/`.

---

3. Mapa de permisos por rol y entidad

App	Entidad	Alumno	Academia	Staff	Propietario
reviews	CourseReview	crear / ver propias	ver	ver	editar propia
reviews	AcademyReview	crear / ver propias	ver	ver	editar propia
payments	Payment	ver propios	ver de sus cursos	ver	—
payments	Discount	ver	crear / gestionar	ver	—
payments	Scholarship	solicitar	aprobar / rechazar	ver	—
notifications	Notification	ver / marcar leídas	ver / marcar leídas	ver	—
evaluations	Evaluation	responder propias	crear / ver de sus cursos	ver	—
talent	TalentPassport	ver propio	ver alumnos matriculados	ver	—
talent	Competence	—	crear / editar	ver	—
level_tests	LevelTest	realizar	crear / editar	ver	—
level_tests	LevelTestQuestion	—	crear / editar	ver	—
search	SearchView	público	público	público	—
dashboard	Dashboard	propio	propio	global	—

---

4. App `reviews`

4.1. Crear la app

```
python manage.py startapp reviews
```

4.2. Modelos

reviews/models.py

```
from django.conf import settings
from django.core.validators import MinValueValidator, MaxValueValidator
from django.db import models

from academies.models import Academy
from courses.models import Course


class CourseReview(models.Model):
    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="course_reviews",
    )
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="reviews",
    )
    rating = models.PositiveSmallIntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(5)],
    )
    comment = models.TextField(blank=True)
    is_published = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["student", "course"],
                name="unique_student_course_review",
            ),
        ]
        indexes = [
            models.Index(fields=["course"]),
            models.Index(fields=["student"]),
        ]

    def __str__(self):
        return f"{self.student.username} → {self.course.title} ({self.rating}/5)"


class AcademyReview(models.Model):
    student = models.ForeignKey(
```

```
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="academy_reviews",
    )
    academy = models.ForeignKey(
        Academy,
        on_delete=models.CASCADE,
        related_name="reviews",
    )
    rating = models.PositiveSmallIntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(5)],
    )
    comment = models.TextField(blank=True)
    is_published = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["student", "academy"],
                name="unique_student_academy_review",
            ),
        ]
        indexes = [
            models.Index(fields=["academy"]),
            models.Index(fields=["student"]),
        ]

    def __str__(self):
        return f"{self.student.username} → {self.academy.name} ({self.rating}/5)"
```

4.3. Formularios

reviews/forms.py

```
from django import forms

from .models import AcademyReview, CourseReview


class CourseReviewForm(forms.ModelForm):
    class Meta:
        model = CourseReview
        fields = ["rating", "comment"]
        widgets = {
            "rating": forms.NumberInput(attrs={"min": 1, "max": 5}),
            "comment": forms.Textarea(attrs={"rows": 4}),
        }
```

```
class AcademyReviewForm(forms.ModelForm):
    class Meta:
        model = AcademyReview
        fields = ["rating", "comment"]
        widgets = {
            "rating": forms.NumberInput(attrs={"min": 1, "max": 5}),
            "comment": forms.Textarea(attrs={"rows": 4}),
        }
```

4.4. Vistas

reviews/views.py

```
import logging

from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404
from django.urls import reverse, reverse_lazy
from django.views.generic import CreateView, DeleteView, ListView, UpdateView

from academies.models import Academy
from courses.models import Course

from .forms import AcademyReviewForm, CourseReviewForm
from .models import AcademyReview, CourseReview

logger = logging.getLogger(__name__)


class StudentRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_student


class CourseReviewCreateView(StudentRequiredMixin, CreateView):
    model = CourseReview
    form_class = CourseReviewForm
    template_name = "reviews/course_review_form.html"

    def get_course(self):
        return get_object_or_404(Course, pk=self.kwargs["course_pk"])

    def form_valid(self, form):
        form.instance.student = self.request.user
        form.instance.course = self.get_course()
        response = super().form_valid(form)
        messages.success(self.request, "Valoración publicada.")
        logger.info(
            "CourseReview created: id=%s student=%s",
            self.object.pk,
            self.request.user.pk,
```

```
        )
        return response

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class CourseReviewUpdateView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = CourseReview
    form_class = CourseReviewForm
    template_name = "reviews/course_review_form.html"

    def test_func(self):
        return self.get_object().student == self.request.user

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class CourseReviewDeleteView(LoginRequiredMixin, UserPassesTestMixin, DeleteView):
    model = CourseReview
    template_name = "reviews/course_review_confirm_delete.html"

    def test_func(self):
        return self.get_object().student == self.request.user

    def get_success_url(self):
        return self.object.course.get_absolute_url()


class AcademyReviewCreateView(StudentRequiredMixin, CreateView):
    model = AcademyReview
    form_class = AcademyReviewForm
    template_name = "reviews/academy_review_form.html"

    def get_academy(self):
        return get_object_or_404(Academy, slug=self.kwargs["slug"])

    def form_valid(self, form):
        form.instance.student = self.request.user
        form.instance.academy = self.get_academy()
        response = super().form_valid(form)
        messages.success(self.request, "Valoración publicada.")
        return response

    def get_success_url(self):
        return reverse("academies:detail", kwargs={"slug": self.object.academy.slug})


class AcademyReviewUpdateView(LoginRequiredMixin, UserPassesTestMixin, UpdateView):
    model = AcademyReview
    form_class = AcademyReviewForm
    template_name = "reviews/academy_review_form.html"
```

```
    def test_func(self):
        return self.get_object().student == self.request.user


class AcademyReviewDeleteView(LoginRequiredMixin, UserPassesTestMixin, DeleteView):
    model = AcademyReview
    template_name = "reviews/academy_review_confirm_delete.html"

    def test_func(self):
        return self.get_object().student == self.request.user


class MyReviewListView(LoginRequiredMixin, ListView):
    template_name = "reviews/my_reviews.html"
    context_object_name = "reviews"

    def get_queryset(self):
        return CourseReview.objects.filter(
            student=self.request.user
        ).select_related("course")
```

4.5. URLs

reviews/urls.py

```
from django.urls import path

from . import views

app_name = "reviews"

urlpatterns = [
    path("mine/", views.MyReviewListView.as_view(), name="mine"),

    path(
        "course/<int:course_pk>/create/",
        views.CourseReviewCreateView.as_view(),
        name="course_create",
    ),
    path(
        "course/<int:pk>/edit/",
        views.CourseReviewUpdateView.as_view(),
        name="course_update",
    ),
    path(
        "course/<int:pk>/delete/",
        views.CourseReviewDeleteView.as_view(),
        name="course_delete",
    ),

    path(
```

```
        "academy/<slug:slug>/create/",
        views.AcademyReviewCreateView.as_view(),
        name="academy_create",
    ),
    path(
        "academy/<int:pk>/edit/",
        views.AcademyReviewUpdateView.as_view(),
        name="academy_update",
    ),
    path(
        "academy/<int:pk>/delete/",
        views.AcademyReviewDeleteView.as_view(),
        name="academy_delete",
    ),
]
```

4.6. Templates

templates/reviews/course_review_form.html

```
{% extends "base.html" %}

{% block title %}
Valorar curso | Academy Hub
{% endblock %}

{% block content %}
<h1>Valorar curso</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Publicar valoración</button>
</form>
{% endblock %}
```

templates/reviews/course_review_confirm_delete.html

```
{% extends "base.html" %}

{% block title %}
Eliminar valoración | Academy Hub
{% endblock %}

{% block content %}
<h1>Eliminar valoración</h1>
<p>¿Seguro que quieres eliminar esta valoración?</p>

<form method="post">
    {% csrf_token %}
    <button type="submit">Sí, eliminar</button>
```

```
</form>
{% endblock %}
```

`templates/reviews/academy_review_form.html` y
`templates/reviews/academy_review_confirm_delete.html` siguen el mismo patrón.

`templates/reviews/my_reviews.html`

```
{% extends "base.html" %}

{% block title %}
Mis valoraciones | Academy Hub
{% endblock %}

{% block content %}
<h1>Mis valoraciones</h1>

{% for review in reviews %}
    <article>
        <p>Curso: {{ review.course.title }}</p>
        <p>Valoración: {{ review.rating }}/5</p>
        <p>{{ review.comment }}</p>

        <a href="{% url 'reviews:course_update' review.pk %}">Editar</a>
        |
        <a href="{% url 'reviews:course_delete' review.pk %}">Eliminar</a>
    </article>
    <hr>
{% empty %}
    <p>Todavía no has hecho ninguna valoración.</p>
{% endfor %}
{% endblock %}
```

4.7. Tests

`reviews/tests.py`

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

from academies.models import Academy
from courses.models import Category, Course, CourseStatus

from .models import AcademyReview, CourseReview

User = get_user_model()


class CourseReviewTests(TestCase):
    def setUp(self):
```

```
        self.academy_user = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.academy = Academy.objects.create(
            owner=self.academy_user, name="English Now", slug="english-now"
        )
        self.category = Category.objects.create(name="Idiomas", slug="idiomas")
        self.course = Course.objects.create(
            academy=self.academy,
            category=self.category,
            title="Inglés B2",
            slug="ingles-b2",
            description="...",
            status=CourseStatus.PUBLISHED,
        )

    def test_student_can_create_review(self):
        self.client.force_login(self.student)
        response = self.client.post(
            reverse("reviews:course_create", kwargs={"course_pk": self.course.pk}),
            {"rating": 5, "comment": "Excelente"},
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(CourseReview.objects.count(), 1)

    def test_student_cannot_review_twice(self):
        self.client.force_login(self.student)
        self.client.post(
            reverse("reviews:course_create", kwargs={"course_pk": self.course.pk}),
            {"rating": 5, "comment": "Excelente"},
        )
        response = self.client.post(
            reverse("reviews:course_create", kwargs={"course_pk": self.course.pk}),
            {"rating": 3, "comment": "Otra"},
        )
        self.assertEqual(CourseReview.objects.count(), 1)

    def test_review_requires_login(self):
        response = self.client.post(
            reverse("reviews:course_create", kwargs={"course_pk": self.course.pk}),
            {"rating": 5},
        )
        self.assertEqual(response.status_code, 302)
```

5. App `payments`

5.1. Crear la app

```
python manage.py startapp payments
```

5.2. Modelos

payments/models.py

```
from django.conf import settings
from django.core.validators import MinValueValidator
from django.db import models

from enrollments.models import Enrollment


class Discount(models.Model):
    class DiscountType(models.TextChoices):
        PERCENTAGE = "percentage", "Porcentaje"
        FIXED = "fixed", "Importe fijo"

    code = models.CharField(max_length=30, unique=True)
    description = models.CharField(max_length=200, blank=True)
    discount_type = models.CharField(
        max_length=20,
        choices=DiscountType.choices,
        default=DiscountType.PERCENTAGE,
    )
    value = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[MinValueValidator(0)],
    )
    valid_from = models.DateField(null=True, blank=True)
    valid_until = models.DateField(null=True, blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.code} ({self.value})"


class Scholarship(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Solicitada"
        APPROVED = "approved", "Aprobada"
        REJECTED = "rejected", "Rechazada"

    student = models.ForeignKey(
```

```
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="scholarships",
    )
    enrollment = models.ForeignKey(
        Enrollment,
        on_delete=models.CASCADE,
        related_name="scholarships",
    )
    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
    amount = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        default=0,
        validators=[MinValueValidator(0)],
    )
    reason = models.TextField(blank=True)
    reviewed_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="reviewed_scholarships",
    )
    reviewed_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"Beca {self.student.username} — {self.status}"


class Payment(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pendiente"
        PAID = "paid", "Pagado"
        FAILED = "failed", "Fallido"
        REFUNDED = "refunded", "Reembolsado"

    enrollment = models.ForeignKey(
        Enrollment,
        on_delete=models.CASCADE,
        related_name="payments",
    )
    amount = models.DecimalField(
        max_digits=10,
        decimal_places=2,
```

```
        validators=[MinValueValidator(0)],
    )
    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
    discount = models.ForeignKey(
        Discount,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="payments",
    )
    reference = models.CharField(max_length=80, blank=True)
    paid_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"Pago {self.enrollment.pk} — {self.amount} ({self.status})"
```

5.3. Formularios

payments/forms.py

```
from django import forms

from .models import Discount, Payment, Scholarship


class DiscountForm(forms.ModelForm):
    class Meta:
        model = Discount
        fields = [
            "code",
            "description",
            "discount_type",
            "value",
            "valid_from",
            "valid_until",
            "is_active",
        ]


class ScholarshipForm(forms.ModelForm):
    class Meta:
        model = Scholarship
        fields = ["reason"]
```

```
class PaymentForm(forms.ModelForm):
    class Meta:
        model = Payment
        fields = ["amount", "discount", "reference"]
```

5.4. Vistas

payments/views.py

```
import logging

from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.urls import reverse_lazy
from django.utils import timezone
from django.views.generic import CreateView, ListView, UpdateView

from .forms import DiscountForm, PaymentForm, ScholarshipForm
from .models import Discount, Payment, Scholarship

logger = logging.getLogger(__name__)


class AcademyRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_academy


class DiscountListView(AcademyRequiredMixin, ListView):
    model = Discount
    template_name = "payments/discount_list.html"
    context_object_name = "discounts"


class DiscountCreateView(AcademyRequiredMixin, CreateView):
    model = Discount
    form_class = DiscountForm
    template_name = "payments/discount_form.html"
    success_url = reverse_lazy("payments:discount_list")

    def form_valid(self, form):
        response = super().form_valid(form)
        messages.success(self.request, "Descuento creado.")
        return response


class DiscountUpdateView(AcademyRequiredMixin, UpdateView):
    model = Discount
    form_class = DiscountForm
    template_name = "payments/discount_form.html"
    success_url = reverse_lazy("payments:discount_list")
```

```
class DiscountDeleteView(AcademyRequiredMixin, UpdateView):
    model = Discount
    form_class = DiscountForm
    template_name = "payments/discount_form.html"
    success_url = reverse_lazy("payments:discount_list")


class StudentScholarshipListView(LoginRequiredMixin, ListView):
    model = Scholarship
    template_name = "payments/scholarship_list.html"
    context_object_name = "scholarships"

    def get_queryset(self):
        return Scholarship.objects.filter(student=self.request.user)


class ScholarshipRequestView(LoginRequiredMixin, CreateView):
    model = Scholarship
    form_class = ScholarshipForm
    template_name = "payments/scholarship_form.html"

    def form_valid(self, form):
        form.instance.student = self.request.user
        response = super().form_valid(form)
        messages.success(self.request, "Solicitud de beca enviada.")
        return response

    def get_success_url(self):
        return reverse_lazy("payments:scholarship_list")


class ScholarshipApproveView(AcademyRequiredMixin, UpdateView):
    model = Scholarship
    fields = ["amount"]
    template_name = "payments/scholarship_approve.html"

    def form_valid(self, form):
        scholarship = form.save(commit=False)
        scholarship.status = Scholarship.Status.APPROVED
        scholarship.reviewed_by = self.request.user
        scholarship.reviewed_at = timezone.now()
        scholarship.save()
        messages.success(self.request, "Beca aprobada.")
        return super().form_valid(form)

    def get_success_url(self):
        return reverse_lazy("payments:scholarship_list")


class PaymentListView(LoginRequiredMixin, ListView):
    model = Payment
```

```
    template_name = "payments/payment_list.html"
    context_object_name = "payments"

    def get_queryset(self):
        user = self.request.user
        if user.is_academy:
            return Payment.objects.filter(
                enrollment__convocatoria__course__academy__owner=user
            )
        return Payment.objects.filter(enrollment__student=user)


class PaymentCreateView(LoginRequiredMixin, CreateView):
    model = Payment
    form_class = PaymentForm
    template_name = "payments/payment_form.html"

    def form_valid(self, form):
        response = super().form_valid(form)
        messages.success(self.request, "Pago registrado.")
        logger.info("Payment created: id=%s", self.object.pk)
        return response

    def get_success_url(self):
        return reverse_lazy("payments:payment_list")
```

5.5. URLs

payments/urls.py

```
from django.urls import path

from . import views

app_name = "payments"

urlpatterns = [
    path("discounts/", views.DiscountListView.as_view(), name="discount_list"),
    path("discounts/create/", views.DiscountCreateView.as_view(), name="discount_create"),
    path("discounts/<int:pk>/edit/", views.DiscountUpdateView.as_view(), name="discount_updat
    path("discounts/<int:pk>/delete/", views.DiscountDeleteView.as_view(), name="discount_del

    path("scholarships/", views.StudentScholarshipListView.as_view(), name="scholarship_list"
    path("scholarships/request/", views.ScholarshipRequestView.as_view(), name="scholarship_r
    path("scholarships/<int:pk>/approve/", views.ScholarshipApproveView.as_view(), name="scho

    path("", views.PaymentListView.as_view(), name="payment_list"),
    path("create/", views.PaymentCreateView.as_view(), name="payment_create"),
]
```

5.6. Templates

- payments/discount_list.html
- payments/discount_form.html
- payments/scholarship_list.html
- payments/scholarship_form.html
- payments/scholarship_approve.html
- payments/payment_list.html
- payments/payment_form.html

Todos siguen el patrón estándar de listado / formulario / confirmación.

5.7. Tests

payments/tests.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

from .models import Discount, Scholarship

User = get_user_model()


class DiscountTests(TestCase):
    def setUp(self):
        self.academy = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )

    def test_academy_can_create_discount(self):
        self.client.force_login(self.academy)
        response = self.client.post(
            reverse("payments:discount_create"),
            {
                "code": "VERANO10",
                "discount_type": "percentage",
                "value": 10,
                "is_active": True,
            },
        )
        self.assertEqual(response.status_code, 302)
        self.assertEqual(Discount.objects.count(), 1)

    def test_student_cannot_create_discount(self):
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
```

```
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("payments:discount_create"))
        self.assertEqual(response.status_code, 403)
```

---

6. App `notifications`

6.1. Crear la app

```
python manage.py startapp notifications
```

6.2. Modelos

notifications/models.py

```
from django.conf import settings
from django.db import models


class Notification(models.Model):
    class Type(models.TextChoices):
        INFO = "info", "Información"
        WARNING = "warning", "Aviso"
        SUCCESS = "success", "Éxito"
        REMINDER = "reminder", "Recordatorio"

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="notifications",
    )
    type = models.CharField(
        max_length=20,
        choices=Type.choices,
        default=Type.INFO,
    )
    title = models.CharField(max_length=200)
    message = models.TextField()
    url = models.CharField(max_length=300, blank=True)
    is_read = models.BooleanField(default=False)
    read_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["user", "is_read"]),
        ]
```

```
    def __str__(self):
        return f"{self.user.username} — {self.title}"
```

6.3. Vistas

notifications/views.py

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.shortcuts import get_object_or_404, redirect
from django.utils import timezone
from django.views.generic import ListView

from .models import Notification


class NotificationListView(LoginRequiredMixin, ListView):
    model = Notification
    template_name = "notifications/notification_list.html"
    context_object_name = "notifications"
    paginate_by = 20

    def get_queryset(self):
        return Notification.objects.filter(user=self.request.user)


def mark_as_read(request, pk):
    notification = get_object_or_404(
        Notification, pk=pk, user=request.user
    )
    if not notification.is_read:
        notification.is_read = True
        notification.read_at = timezone.now()
        notification.save()
    return redirect("notifications:list")


def mark_all_as_read(request):
    Notification.objects.filter(
        user=request.user, is_read=False
    ).update(is_read=True, read_at=timezone.now())
    return redirect("notifications:list")
```

6.4. URLs

notifications/urls.py

```
from django.urls import path

from . import views
```

```
app_name = "notifications"

urlpatterns = [
    path("", views.NotificationListView.as_view(), name="list"),
    path("<int:pk>/read/", views.mark_as_read, name="mark_as_read"),
    path("read-all/", views.mark_all_as_read, name="mark_all_as_read"),
]
```

6.5. Templates

notifications/notification_list.html

```
{% extends "base.html" %}

{% block title %}
Notificaciones | Academy Hub
{% endblock %}

{% block content %}
<h1>Notificaciones</h1>

<p>
    <a href="{% url 'notifications:mark_all_as_read' %}">
        Marcar todas como leídas
    </a>
</p>

{% for notification in notifications %}
    <article>
        <h2>{{ notification.title }}</h2>
        <p>{{ notification.message }}</p>
        <p>{{ notification.created_at }}</p>

        {% if not notification.is_read %}
            <a href="{% url 'notifications:mark_as_read' notification.pk %}">
                Marcar como leída
            </a>
        {% endif %}
    </article>
    <hr>
{% empty %}
    <p>No tienes notificaciones.</p>
{% endfor %}
{% endblock %}
```

6.6. Tests

notifications/tests.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase
```

```
from django.urls import reverse

from .models import Notification

User = get_user_model()


class NotificationTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
        )
        self.notification = Notification.objects.create(
            user=self.user, title="Hola", message="Mensaje"
        )

    def test_user_sees_only_own_notifications(self):
        other = User.objects.create_user(
            username="carlos",
            email="carlos@example.com",
            password="TestPassword123!",
        )
        Notification.objects.create(user=other, title="Otro", message="...")

        self.client.force_login(self.user)
        response = self.client.get(reverse("notifications:list"))
        self.assertContains(response, "Hola")
        self.assertNotContains(response, "Otro")

    def test_mark_as_read(self):
        self.client.force_login(self.user)
        self.client.get(
            reverse("notifications:mark_as_read", kwargs={"pk": self.notification.pk})
        )
        self.notification.refresh_from_db()
        self.assertTrue(self.notification.is_read)
```

---

7. App `evaluations`

7.1. Crear la app

```
python manage.py startapp evaluations
```

7.2. Modelos

evaluations/models.py

```
from django.conf import settings
from django.db import models

from enrollments.models import Enrollment


class Evaluation(models.Model):
    class Kind(models.TextChoices):
        INITIAL = "initial", "Inicial"
        FINAL = "final", "Final"

    enrollment = models.ForeignKey(
        Enrollment,
        on_delete=models.CASCADE,
        related_name="evaluations",
    )
    kind = models.CharField(max_length=20, choices=Kind.choices)
    score = models.DecimalField(max_digits=5, decimal_places=2, null=True, blank=True)
    feedback = models.TextField(blank=True)
    evaluated_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="evaluations_made",
    )
    evaluated_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["enrollment", "kind"],
                name="unique_evaluation_per_enrollment_kind",
            ),
        ]

    def __str__(self):
        return f"{self.enrollment} — {self.kind}"
```

7.3. Formularios

evaluations/forms.py

```
from django import forms

from .models import Evaluation


class EvaluationForm(forms.ModelForm):
    class Meta:
```

```
        model = Evaluation
        fields = ["score", "feedback"]
```

7.4. Vistas

evaluations/views.py

```
from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404
from django.urls import reverse_lazy
from django.utils import timezone
from django.views.generic import CreateView, ListView, UpdateView

from enrollments.models import Enrollment

from .forms import EvaluationForm
from .models import Evaluation


class AcademyRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_academy


class EvaluationListView(AcademyRequiredMixin, ListView):
    model = Evaluation
    template_name = "evaluations/evaluation_list.html"
    context_object_name = "evaluations"

    def get_queryset(self):
        return Evaluation.objects.filter(
            enrollment__convocatoria__course__academy__owner=self.request.user
        ).select_related("enrollment", "enrollment__student")


class EvaluationCreateView(AcademyRequiredMixin, CreateView):
    model = Evaluation
    form_class = EvaluationForm
    template_name = "evaluations/evaluation_form.html"

    def get_enrollment(self):
        return get_object_or_404(Enrollment, pk=self.kwargs["enrollment_pk"])

    def form_valid(self, form):
        form.instance.enrollment = self.get_enrollment()
        form.instance.evaluated_by = self.request.user
        form.instance.evaluated_at = timezone.now()
        response = super().form_valid(form)
        messages.success(self.request, "Evaluación registrada.")
        return response
```

```
    def get_success_url(self):
        return reverse_lazy("evaluations:list")


class EvaluationUpdateView(AcademyRequiredMixin, UpdateView):
    model = Evaluation
    form_class = EvaluationForm
    template_name = "evaluations/evaluation_form.html"
    success_url = reverse_lazy("evaluations:list")

    def form_valid(self, form):
        form.instance.evaluated_by = self.request.user
        form.instance.evaluated_at = timezone.now()
        return super().form_valid(form)
```

7.5. URLs

evaluations/urls.py

```
from django.urls import path

from . import views

app_name = "evaluations"

urlpatterns = [
    path("", views.EvaluationListView.as_view(), name="list"),
    path(
        "enrollment/<int:enrollment_pk>/create/",
        views.EvaluationCreateView.as_view(),
        name="create",
    ),
    path("<int:pk>/edit/", views.EvaluationUpdateView.as_view(), name="update"),
]
```

7.6. Templates

- evaluations/evaluation_list.html
- evaluations/evaluation_form.html

7.7. Tests

evaluations/tests.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

User = get_user_model()
```

```
class EvaluationTests(TestCase):
    def test_academy_can_see_evaluations(self):
        academy = User.objects.create_user(
            username="englishnow",
            email="info@englishnow.com",
            password="TestPassword123!",
            is_academy=True,
        )
        self.client.force_login(academy)
        response = self.client.get(reverse("evaluations:list"))
        self.assertEqual(response.status_code, 200)

    def test_student_cannot_see_evaluations(self):
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("evaluations:list"))
        self.assertEqual(response.status_code, 403)
```

---

8. App `talent`

8.1. Crear la app

```
python manage.py startapp talent
```

8.2. Modelos

talent/models.py

```
from django.conf import settings
from django.db import models


class Competence(models.Model):
    name = models.CharField(max_length=120, unique=True)
    description = models.TextField(blank=True)
    category = models.ForeignKey(
        "courses.Category",
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="competences",
    )
    is_active = models.BooleanField(default=True)
```

```
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name


class TalentPassport(models.Model):
    student = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="talent_passport",
    )
    headline = models.CharField(max_length=200, blank=True)
    summary = models.TextField(blank=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Pasaporte de {self.student.username}"


class PassportEntry(models.Model):
    passport = models.ForeignKey(
        TalentPassport,
        on_delete=models.CASCADE,
        related_name="entries",
    )
    competence = models.ForeignKey(
        Competence,
        on_delete=models.CASCADE,
        related_name="entries",
    )
    level = models.PositiveSmallIntegerField(default=1)
    evidence = models.TextField(blank=True)
    obtained_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-obtained_at"]
        constraints = [
            models.UniqueConstraint(
                fields=["passport", "competence"],
                name="unique_passport_competence",
            ),
        ]

    def __str__(self):
        return f"{self.passport.student.username} — {self.competence.name}"
```

8.3. Formularios

talent/forms.py

```
from django import forms

from .models import Competence, PassportEntry, TalentPassport


class CompetenceForm(forms.ModelForm):
    class Meta:
        model = Competence
        fields = ["name", "description", "category", "is_active"]


class PassportEntryForm(forms.ModelForm):
    class Meta:
        model = PassportEntry
        fields = ["competence", "level", "evidence"]


class TalentPassportForm(forms.ModelForm):
    class Meta:
        model = TalentPassport
        fields = ["headline", "summary"]
```

8.4. Vistas

talent/views.py

```
from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404
from django.urls import reverse_lazy
from django.views.generic import CreateView, DetailView, ListView, UpdateView

from .forms import CompetenceForm, PassportEntryForm, TalentPassportForm
from .models import Competence, TalentPassport


class StaffRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_staff


class CompetenceListView(StaffRequiredMixin, ListView):
    model = Competence
    template_name = "talent/competence_list.html"
    context_object_name = "competences"


class CompetenceCreateView(StaffRequiredMixin, CreateView):
    model = Competence
    form_class = CompetenceForm
    template_name = "talent/competence_form.html"
    success_url = reverse_lazy("talent:competence_list")
```

```
class CompetenceUpdateView(StaffRequiredMixin, UpdateView):
    model = Competence
    form_class = CompetenceForm
    template_name = "talent/competence_form.html"
    success_url = reverse_lazy("talent:competence_list")


class MyPassportView(LoginRequiredMixin, DetailView):
    template_name = "talent/passport_detail.html"
    context_object_name = "passport"

    def get_object(self):
        passport, _ = TalentPassport.objects.get_or_create(
            student=self.request.user
        )
        return passport


class MyPassportEditView(LoginRequiredMixin, UpdateView):
    model = TalentPassport
    form_class = TalentPassportForm
    template_name = "talent/passport_form.html"
    success_url = reverse_lazy("talent:passport")

    def get_object(self):
        passport, _ = TalentPassport.objects.get_or_create(
            student=self.request.user
        )
        return passport


class PassportEntryCreateView(LoginRequiredMixin, UserPassesTestMixin, CreateView):
    form_class = PassportEntryForm
    template_name = "talent/entry_form.html"

    def test_func(self):
        return self.request.user.is_academy

    def get_passport(self):
        from django.contrib.auth import get_user_model
        User = get_user_model()
        student = get_object_or_404(User, pk=self.kwargs["student_pk"])
        passport, _ = TalentPassport.objects.get_or_create(student=student)
        return passport

    def form_valid(self, form):
        form.instance.passport = self.get_passport()
        response = super().form_valid(form)
        messages.success(self.request, "Competencia añadida al pasaporte.")
        return response
```

```
    def get_success_url(self):
        return reverse_lazy("talent:passport")
```

8.5. URLs

talent/urls.py

```
from django.urls import path

from . import views

app_name = "talent"

urlpatterns = [
    path("", views.MyPassportView.as_view(), name="passport"),
    path("edit/", views.MyPassportEditView.as_view(), name="passport_edit"),

    path("competences/", views.CompetenceListView.as_view(), name="competence_list"),
    path("competences/create/", views.CompetenceCreateView.as_view(), name="competence_create
    path("competences/<int:pk>/edit/", views.CompetenceUpdateView.as_view(), name="competence

    path(
        "student/<int:student_pk>/add-entry/",
        views.PassportEntryCreateView.as_view(),
        name="entry_create",
    ),
]
```

8.6. Templates

- talent/passport_detail.html
- talent/passport_form.html
- talent/competence_list.html
- talent/competence_form.html
- talent/entry_form.html

8.7. Tests

talent/tests.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

User = get_user_model()


class PassportTests(TestCase):
    def test_student_can_see_own_passport(self):
```

```
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("talent:passport"))
        self.assertEqual(response.status_code, 200)

    def test_competences_require_staff(self):
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("talent:competence_list"))
        self.assertEqual(response.status_code, 403)
```

---

9. App `level_tests`

9.1. Crear la app

```
python manage.py startapp level_tests
```

9.2. Modelos

level_tests/models.py

```
from django.conf import settings
from django.db import models

from courses.models import Course


class LevelTest(models.Model):
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="level_tests",
    )
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    passing_score = models.PositiveSmallIntegerField(default=60)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    def __str__(self):
        return f"{self.course.title} — {self.title}"


class LevelTestQuestion(models.Model):
    level_test = models.ForeignKey(
        LevelTest,
        on_delete=models.CASCADE,
        related_name="questions",
    )
    question = models.CharField(max_length=300)
    option_a = models.CharField(max_length=200)
    option_b = models.CharField(max_length=200)
    option_c = models.CharField(max_length=200, blank=True)
    option_d = models.CharField(max_length=200, blank=True)
    correct_option = models.CharField(max_length=1, default="A")

    def __str__(self):
        return self.question


class LevelTestAttempt(models.Model):
    student = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="level_test_attempts",
    )
    level_test = models.ForeignKey(
        LevelTest,
        on_delete=models.CASCADE,
        related_name="attempts",
    )
    score = models.PositiveSmallIntegerField(default=0)
    passed = models.BooleanField(default=False)
    started_at = models.DateTimeField(auto_now_add=True)
    finished_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ["-started_at"]

    def __str__(self):
        return f"{self.student.username} — {self.level_test.title}"
```

9.3. Vistas

level_tests/views.py

```
from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.shortcuts import get_object_or_404, redirect
from django.urls import reverse_lazy
from django.utils import timezone
```

```
from django.views.generic import CreateView, DetailView, ListView, UpdateView

from .forms import LevelTestAttemptForm, LevelTestForm
from .models import LevelTest, LevelTestAttempt


class AcademyRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_academy


class LevelTestListView(ListView):
    model = LevelTest
    template_name = "level_tests/level_test_list.html"
    context_object_name = "level_tests"

    def get_queryset(self):
        return LevelTest.objects.filter(is_active=True).select_related("course")


class LevelTestDetailView(DetailView):
    model = LevelTest
    template_name = "level_tests/level_test_detail.html"
    context_object_name = "level_test"


class LevelTestCreateView(AcademyRequiredMixin, CreateView):
    model = LevelTest
    form_class = LevelTestForm
    template_name = "level_tests/level_test_form.html"
    success_url = reverse_lazy("level_tests:list")


class LevelTestUpdateView(AcademyRequiredMixin, UpdateView):
    model = LevelTest
    form_class = LevelTestForm
    template_name = "level_tests/level_test_form.html"
    success_url = reverse_lazy("level_tests:list")


class LevelTestAttemptView(LoginRequiredMixin, CreateView):
    model = LevelTestAttempt
    form_class = LevelTestAttemptForm
    template_name = "level_tests/attempt_form.html"

    def get_level_test(self):
        return get_object_or_404(LevelTest, pk=self.kwargs["pk"])

    def form_valid(self, form):
        form.instance.student = self.request.user
        form.instance.level_test = self.get_level_test()
        response = super().form_valid(form)
        messages.success(self.request, "Prueba de nivel registrada.")
```

```
        return response

    def get_success_url(self):
        return reverse_lazy("level_tests:list")
```

9.4. Formularios

level_tests/forms.py

```
from django import forms

from .models import LevelTest, LevelTestAttempt


class LevelTestForm(forms.ModelForm):
    class Meta:
        model = LevelTest
        fields = ["course", "title", "description", "passing_score", "is_active"]


class LevelTestAttemptForm(forms.ModelForm):
    class Meta:
        model = LevelTestAttempt
        fields = ["score", "passed"]
```

9.5. URLs

level_tests/urls.py

```
from django.urls import path

from . import views

app_name = "level_tests"

urlpatterns = [
    path("", views.LevelTestListView.as_view(), name="list"),
    path("create/", views.LevelTestCreateView.as_view(), name="create"),
    path("<int:pk>/", views.LevelTestDetailView.as_view(), name="detail"),
    path("<int:pk>/edit/", views.LevelTestUpdateView.as_view(), name="update"),
    path("<int:pk>/attempt/", views.LevelTestAttemptView.as_view(), name="attempt"),
]
```

9.6. Templates

- level_tests/level_test_list.html
- level_tests/level_test_detail.html
- level_tests/level_test_form.html
- level_tests/attempt_form.html

9.7. Tests

level_tests/tests.py

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

User = get_user_model()


class LevelTestTests(TestCase):
    def test_list_is_public(self):
        response = self.client.get(reverse("level_tests:list"))
        self.assertEqual(response.status_code, 200)

    def test_create_requires_academy(self):
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("level_tests:create"))
        self.assertEqual(response.status_code, 403)
```

---

10. App `search`

10.1. Crear la app

```
python manage.py startapp search
```

10.2. Formularios

search/forms.py

```
from django import forms


class CourseSearchForm(forms.Form):
    q = forms.CharField(required=False, label="Buscar")
    category = forms.IntegerField(required=False)
    level = forms.CharField(required=False)
    modality = forms.CharField(required=False)
    max_price = forms.DecimalField(required=False, min_value=0)
```

10.3. Vistas

search/views.py

```
from django.db.models import Min, Q
from django.views.generic import ListView

from courses.models import Category, Course, CourseLevel, CourseModality, CourseStatus


class CourseSearchView(ListView):
    template_name = "search/course_search.html"
    context_object_name = "courses"
    paginate_by = 12

    def get_queryset(self):
        queryset = (
            Course.objects
            .filter(status=CourseStatus.PUBLISHED)
            .select_related("academy", "category")
        )

        q = self.request.GET.get("q")
        if q:
            queryset = queryset.filter(
                Q(title__icontains=q)
                | Q(description__icontains=q)
                | Q(academy__name__icontains=q)
            )

        category = self.request.GET.get("category")
        if category:
            queryset = queryset.filter(category_id=category)

        level = self.request.GET.get("level")
        if level:
            queryset = queryset.filter(level=level)

        modality = self.request.GET.get("modality")
        if modality:
            queryset = queryset.filter(modality=modality)

        return queryset

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["categories"] = Category.objects.filter(is_active=True)
        context["levels"] = CourseLevel.choices
        context["modalities"] = CourseModality.choices
        context["current_q"] = self.request.GET.get("q", "")
        context["current_category"] = self.request.GET.get("category", "")
        context["current_level"] = self.request.GET.get("level", "")
```

```
        context["current_modality"] = self.request.GET.get("modality", "")
        return context
```

10.4. URLs

search/urls.py

```
from django.urls import path

from . import views

app_name = "search"

urlpatterns = [
    path("", views.CourseSearchView.as_view(), name="course_search"),
]
```

10.5. Templates

templates/search/course_search.html

```
{% extends "base.html" %}

{% block title %}
Buscar cursos | Academy Hub
{% endblock %}

{% block content %}
<h1>Buscar cursos</h1>

<form method="get">
    <input type="text" name="q" value="{{ current_q }}" placeholder="Buscar...">

    <select name="category">
        <option value="">Todas las categorías</option>
        {% for cat in categories %}
            <option value="{{ cat.pk }}" {% if current_category == cat.pk|stringformat:"s" %}
                {{ cat.name }}
            </option>
        {% endfor %}
    </select>

    <select name="level">
        <option value="">Cualquier nivel</option>
        {% for value, label in levels %}
            <option value="{{ value }}" {% if current_level == value %}selected{% endif %}>
                {{ label }}
            </option>
        {% endfor %}
    </select>
```

```
    <select name="modality">
        <option value="">Cualquier modalidad</option>
        {% for value, label in modalities %}
            <option value="{{ value }}" {% if current_modality == value %}selected{% endif %}
                {{ label }}
            </option>
        {% endfor %}
    </select>

    <button type="submit">Buscar</button>
</form>

{% for course in courses %}
    <article>
        <h2>
            <a href="{{ course.get_absolute_url }}">
                {{ course.title }}
            </a>
        </h2>
        <p>{{ course.academy.name }}</p>
        <p>{{ course.get_level_display }}</p>
        <p>{{ course.get_modality_display }}</p>
    </article>
    <hr>
{% empty %}
    <p>No se encontraron cursos.</p>
{% endfor %}
{% endblock %}
```

10.6. Tests

search/tests.py

```
from django.test import TestCase
from django.urls import reverse


class SearchTests(TestCase):
    def test_search_page_is_public(self):
        response = self.client.get(reverse("search:course_search"))
        self.assertEqual(response.status_code, 200)
```

---

11. App `dashboard`

11.1. Crear la app

```
python manage.py startapp dashboard
```

11.2. Vistas

dashboard/views.py

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import TemplateView

from courses.models import Course
from enrollments.models import Enrollment
from payments.models import Payment
from reviews.models import CourseReview


class DashboardView(LoginRequiredMixin, TemplateView):
    def get_template_names(self):
        user = self.request.user
        if user.is_staff:
            return ["dashboard/staff_dashboard.html"]
        if user.is_academy:
            return ["dashboard/academy_dashboard.html"]
        return ["dashboard/student_dashboard.html"]

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        user = self.request.user

        if user.is_staff:
            context["total_courses"] = Course.objects.count()
            context["total_enrollments"] = Enrollment.objects.count()

        elif user.is_academy:
            context["my_courses"] = Course.objects.filter(academy__owner=user)
            context["my_enrollments"] = Enrollment.objects.filter(
                convocatoria__course__academy__owner=user
            )

        else:
            context["my_enrollments"] = Enrollment.objects.filter(student=user)
            context["my_payments"] = Payment.objects.filter(enrollment__student=user)
            context["my_reviews"] = CourseReview.objects.filter(student=user)

        return context
```

11.3. URLs

dashboard/urls.py

```
from django.urls import path

from . import views
```

```
app_name = "dashboard"

urlpatterns = [
    path("", views.DashboardView.as_view(), name="home"),
]
```

11.4. Templates

- `templates/dashboard/student_dashboard.html`
- `templates/dashboard/academy_dashboard.html`
- `templates/dashboard/staff_dashboard.html`

Cada uno muestra un resumen adaptado al rol.

11.5. Tests

`dashboard/tests.py`

```
from django.contrib.auth import get_user_model
from django.test import TestCase
from django.urls import reverse

User = get_user_model()


class DashboardTests(TestCase):
    def test_requires_login(self):
        response = self.client.get(reverse("dashboard:home"))
        self.assertEqual(response.status_code, 302)

    def test_student_sees_student_dashboard(self):
        student = User.objects.create_user(
            username="laura",
            email="laura@example.com",
            password="TestPassword123!",
            is_student=True,
        )
        self.client.force_login(student)
        response = self.client.get(reverse("dashboard:home"))
        self.assertTemplateUsed(response, "dashboard/student_dashboard.html")
```

---

12. Registro en settings y urls

`config/settings/base.py`

```
LOCAL_APPS = [
    "core",
    "accounts",
```

```
    "academies",
    "courses",
    "enrollments",
    "reviews",
    "payments",
    "notifications",
    "evaluations",
    "talent",
    "level_tests",
    "search",
    "dashboard",
]
```

config/urls.py

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("core.urls")),
    path("accounts/", include("accounts.urls")),
    path("academies/", include("academies.urls")),
    path("courses/", include("courses.urls")),
    path("enrollments/", include("enrollments.urls")),
    path("reviews/", include("reviews.urls")),
    path("payments/", include("payments.urls")),
    path("notifications/", include("notifications.urls")),
    path("evaluations/", include("evaluations.urls")),
    path("talent/", include("talent.urls")),
    path("level-tests/", include("level_tests.urls")),
    path("search/", include("search.urls")),
    path("dashboard/", include("dashboard.urls")),
]
```

Migraciones

```
python manage.py makemigrations
python manage.py migrate
python manage.py check
```

---

13. Tabla de URLs finales

```
/reviews/mine/
/reviews/course/<course_pk>/create/
/reviews/course/<pk>/edit/
/reviews/course/<pk>/delete/
/reviews/academy/<slug>/create/
/reviews/academy/<pk>/edit/
/reviews/academy/<pk>/delete/
```

```
/payments/discounts/
/payments/discounts/create/
/payments/discounts/<pk>/edit/
/payments/discounts/<pk>/delete/
/payments/scholarships/
/payments/scholarships/request/
/payments/scholarships/<pk>/approve/
/payments/
/payments/create/

/notifications/
/notifications/<pk>/read/
/notifications/read-all/

/evaluations/
/evaluations/enrollment/<enrollment_pk>/create/
/evaluations/<pk>/edit/

/talent/
/talent/edit/
/talent/competences/
/talent/competences/create/
/talent/competences/<pk>/edit/
/talent/student/<student_pk>/add-entry/

/level-tests/
/level-tests/create/
/level-tests/<pk>/
/level-tests/<pk>/edit/
/level-tests/<pk>/attempt/

/search/

/dashboard/
```

---

14. Tabla de permisos final

Ruta	Visitante	Alumno	Academia	Staff
reviews:course_create	—		—	—
reviews:course_update	—	(propia)	—	—
reviews:academy_create	—		—	—
payments:discount_list	—	—		—
payments:scholarship_list	—		—	—
payments:scholarship_request	—		—	—
payments:scholarship_approve	—	—		—
payments:payment_list	—	(propios)	(sus cursos)	—
payments:payment_create	—		—	—
notifications:list	—			
evaluations:list	—	—		—
evaluations:create	—	—		—

Ruta	Visitante	Alumno	Academia	Staff
talent:passport	—			
talent:competence_list	—	—	—	
talent:entry_create	—	—		—
level_tests:list				
level_tests:create	—	—		—
level_tests:attempt	—		—	—
search:course_search				
dashboard:home	—	(alumno)	(academia)	(staff)

---

15. Checklist global

`reviews`

- [x] Modelos `CourseReview` y `AcademyReview`.
- [x] Formularios.
- [x] Vistas CBV.
- [x] URLs.
- [x] Templates.
- [x] Tests.

`payments`

- [x] Modelos `Discount`, `Scholarship`, `Payment`.
- [x] Formularios.
- [x] Vistas CBV.
- [x] URLs.
- [x] Templates.
- [x] Tests.

`notifications`

- [x] Modelo `Notification`.
- [x] Vista de listado y acciones de marcar leídas.
- [x] URLs.
- [x] Templates.
- [x] Tests.

`evaluations`

- [x] Modelo `Evaluation`.
- [x] Formulario.
- [x] Vistas CBV.
- [x] URLs.
- [x] Templates.

- [x] Tests.

`talent`

- [x] Modelos `Competence`, `TalentPassport`, `PassportEntry`.
- [x] Formularios.
- [x] Vistas CBV.
- [x] URLs.
- [x] Templates.
- [x] Tests.

`level_tests`

- [x] Modelos `LevelTest`, `LevelTestQuestion`, `LevelTestAttempt`.
- [x] Formularios.
- [x] Vistas CBV.
- [x] URLs.
- [x] Templates.
- [x] Tests.

`search`

- [x] Formulario `CourseSearchForm`.
- [x] Vista `CourseSearchView`.
- [x] URL.
- [x] Template.
- [x] Tests.

`dashboard`

- [x] Vista `DashboardView` por rol.
- [x] URL.
- [x] Templates.
- [x] Tests.

Global

- [x] Registro en `INSTALLED_APPS`.
- [x] `include()` en `config/urls.py`.
- [x] Migraciones.
- [x] `python manage.py check` sin errores.
- [x] Tests OK.

---

16. Orden de implementación recomendado

1. `reviews` (depende solo de `courses`, `academies`, `accounts`).
2. `notifications` (independiente).
3. `payments` (depende de `enrollments`).
4. `evaluations` (depende de `enrollments`).
5. `talent` (depende de `accounts` y `courses`).
6. `level_tests` (depende de `courses`).
7. `search` (depende de `courses`).
8. `dashboard` (depende de todas).

Cada vez que se añada una app:

```
python manage.py makemigrations <app>
python manage.py migrate
python manage.py check
python manage.py test
```

---

17. Commit recomendado

```
git status
git add .
git commit -m "feat: add reviews, payments, notifications, evaluations, talent, level_tests,
```

---

18. Notas de integración con apps existentes

- `reviews`: en la vista de detalle de `Course` y en la de `Academy`, se mostrarán las valoraciones publicadas, y un botón "Valorar" si el usuario es alumno.
- `payments`: desde el detalle de `Enrollment`, la academia y el alumno podrán gestionar los pagos. La vista de `Payment` filtra por rol.
- `notifications`: se poblará automáticamente desde eventos de otras apps (reservas próximas a caducar, documentación pendiente, aprobación de becas, etc.). En esta sesión se modela el modelo y la UI; la generación automática se conecta en sesiones posteriores con Celery y Signals.
- `evaluations`: se enlazan a `Enrollment`. Un `Enrollment` puede tener dos `Evaluation`: `initial` y `final`.
- `talent`: el pasaporte se crea bajo demanda (`get_or_create`). Las competencias las gestiona el staff. Las entradas al pasaporte las puede crear una academia al evaluar a un alumno.
- `level_tests`: se enlazan a `Course`. La prueba de nivel se ofrece al alumno antes de matricularse, y su resultado se guarda en `LevelTestAttempt`.
- `search`: la vista se enlaza desde el navbar y desde la página de cursos.
- `dashboard`: se convierte en la página principal tras el login, adaptada al rol del usuario.

Resultado final de la Sesión 6

Academy Hub queda completo funcionalmente:

- Un alumno puede descubrir cursos, inscribirse, pagar, subir documentos, hacer prueba de nivel, recibir notificaciones, ver sus evaluaciones, construir su Pasaporte de Talento y valorar cursos y academias.
- Una academia puede gestionar cursos, convocatorias, imágenes, descuentos, becas, solicitudes, matrículas, evaluaciones y competencias.
- El staff puede gestionar categorías, competencias y ver mensajes de contacto.
- Un dashboard unificado para cada rol.
- Un buscador avanzado con filtros.

Con esto, el diseño original de Academy Hub —"cubrir todo el recorrido, desde descubrir un curso hasta reservar y matricularse"— queda implementado de principio a fin.